Axiomander + Pydantic: Recommended Shape System Architecture

Executive Summary

Axiomander should adopt **Pydantic as its primary source of object and data shape information**, while maintaining its own internal logical representation of shapes and invariants.

Pydantic should be treated as a frontend language for shape specifications, not as the verifier's logical foundation.

The architecture should be:

```
Python
  ↓
Pydantic Models
  ↓
Axiomander Shape IR
  ↓
Verification Conditions
  ↓
SMT / Rocq / LLM Oracle
```

This approach provides:

- Immediate compatibility with existing Python ecosystems
- Runtime validation
- JSON schema generation
- Strong type information
- A migration path toward richer formal specifications
- Minimal additional burden on developers

Design Principle

Do not invent a new shape language until there is a proven need.

Most modern Python applications already describe their data using:

- Pydantic
- FastAPI request models

- Settings models
- API schemas

Axiomander should leverage this existing ecosystem.

The verifier should understand Pydantic models directly.

Core Architecture

User View

The user writes:

```
class User(BaseModel):
    id: int
    email: str
    active: bool
```

and can immediately use:

```
def activate_user(user: User):
    ...
```

without writing any additional contracts.

Internal View

Axiomander lowers the model into a Shape IR.

Example:

```
Shape(User):
    field(id, int)
    field(email, str)
    field(active, bool)
```

This Shape IR becomes the canonical representation used by:

- SMT encodings
- Rocq encodings

- Symbolic execution
- Contract checking

Pydantic itself is never the proof language.

Shape Logic

Every model should compile into a small logical language.

Field Presence

```
has_field(user, "email")
```

Field Type

```
field_type(user, "email", str)
```

Field Value

```
field_value(user, "email") = value
```

Constraint Lowering

Pydantic constraints should become logical predicates.

Example:

```
amount: Annotated[int, Field(gt=0)]
```

becomes:

```
amount : int
amount > 0
```

Numeric Constraints

Pydantic:

```
Field(gt=0)
Field(ge=0)
Field(lt=100)
Field(le=100)
```

IR:

```
x > 0
x >= 0
x < 100
x <= 100
```

These are ideal SMT obligations.

String Constraints

Pydantic:

```
Field(min_length=3)
Field(max_length=20)
```

IR:

```
len(x) >= 3
len(x) <= 20
```

Literal Types

Pydantic:

```
Literal["pending", "paid", "cancelled"]
```

IR:

```
status ∈ {
    pending,
    paid,
    cancelled
}
```

Nested Models

Example:

```
class Address(BaseModel):
    city: str

class User(BaseModel):
    address: Address
```

IR:

```
Shape(User):
    address : Shape(Address)
```

The verifier should recursively expand nested shapes.

Collections

Lists

```
list[int]
```

IR:

```
forall i:
    list[i] : int
```

Optional:

```
len(list) = n
```

Dictionaries

```
dict[str, int]
```

IR:

```
forall key:
    key : str

forall value:
    value : int
```

Optional Values

Pydantic:

```
Optional[str]
```

IR:

```
email = None
OR
email : str
```

This should integrate naturally with SMT encodings.

Union Types

Pydantic:

```
Union[A, B]
```

IR:

```
shape(A)
OR
shape(B)
```

Initially this can be represented as a tagged disjunction.

Validators

Validators require careful treatment.

Category 1: Trivial Pure Validators

Example:

```
@field_validator("amount")
def positive(v):
    assert v > 0
    return v
```

Lower directly into logical predicates.

Category 2: Pure Complex Validators

Example:

```
@field_validator("email")
def valid_email(v):
    return complicated_regex(v)
```

Represent as:

```
valid_email(v)
```

and generate proof obligations where possible.

Initially these may remain uninterpreted predicates.

Category 3: Effectful Validators

Example:

```
@field_validator("username")
def unique(v):
    db.lookup(v)
```

These should not enter the logic.

Treat them as trusted assumptions:

```
Assume(unique_username(v))
```

and emit verification warnings.

Category 4: Transforming Validators

Example:

```
@field_validator("email")
def normalize(v):
    return v.lower()
```

Represent as:

```
email_out = lowercase(email_in)
```

This becomes a postcondition.

Database Integration

Pydantic models should become the natural bridge between:

```
Database Rows
API Requests
```

```
Domain Objects
Verification Shapes
```

Example:

```
class UserRow(BaseModel):
    id: int
    email: str
```

Axiomander can associate:

```
db.users row shape = UserRow
```

This immediately provides:

- Schema reasoning
- Row shape reasoning
- Query result typing

without inventing a separate schema language.

Function Contracts

Shape information should automatically become function contracts.

Example:

```
def create_user(req: CreateUserRequest):
    ...
```

Automatically implies:

```
Precondition:
    req satisfies Shape(CreateUserRequest)
```

No extra user annotation required.

Recommended Verification Strategy

Use a layered approach.

Layer 1

Runtime Pydantic validation.

Guarantees:

```
Incoming data matches declared shape.
```

Layer 2

Axiomander shape reasoning.

Guarantees:

```
Program preserves shape invariants.
```

Layer 3

SMT discharge.

Guarantees:

```
Numeric
Boolean
Collection
Shape constraints
```

Layer 4

Rocq proofs.

Guarantees:

```
Recursive properties
Inductive properties
Complex invariants
```

Long-Term Goal

The eventual goal is:

```
Pydantic
    ↓
Shape IR
    ↓
Heap Regions
    ↓
Database Regions
    ↓
Ownership Logic
    ↓
Concurrent Protocols
```

The Pydantic integration is not the end-state.

It is the most practical route toward realistic verification of Python software while preserving compatibility with existing ecosystems.

Final Recommendation

Implement an `axiomander-pydantic` frontend package.

Phase 1:

- Parse BaseModel definitions
- Extract fields
- Extract constraints
- Generate Shape IR

Phase 2:

- Support nested models
- Support collections

- Support Optionals
- Support Literals

Phase 3:

- Support validators
- Generate logical predicates
- Generate assumptions where necessary

Phase 4:

- Integrate shapes with heap regions
- Integrate shapes with database regions
- Generate frame conditions automatically

The verifier should reason about Axiomander Shape IR, not Pydantic directly.